

Video Transcript

Video 3: NiFi: An Apache ETL Tool (04:10)

We will once again use an excellent project from the Apache Software Foundation, in this case called NiFi, and this project is very well suited for ETL. As you can see here the project description, their definition is, scalable directed graphs for data routing, transformation, and system mediation logic. This is a fancy way of saying my definition, data pipelines and transformation on the data flowing through that pipeline. Now when it comes to features, this is pretty good at handling pipelines where you're doing conversions or you're going between systems. It has a pretty friendly user interface as we will see, and that it's modelled around processors. You can think of the processors as transformations.

It is also pretty scalable. You can simply add machines to it to do horizontal scale. That is, you simply add more commodity machines, and you will be able to get more capacity. Now when it comes to the fit, what can it do well? You can see here that it can transfer or that is make mappings onto many different systems. And this is one of the things that you want to do when you're doing ETL. You want to be able to extract and then distribute. As you can see there, that's precisely that second point, which means that it can deliver to analytic platforms.

That's typically the case where you want to act upon the data. And then, you can enrich and prepare further for subsequent steps. Now, what is it not good at? It's certainly not a distributed computation platform. So, if you're trying to do high performance computing, this is not the platform to use. Likewise, it is not for complex event processing. And if you're thinking about doing aggregate operations on large volumes of data, that once again, it's not what this is for. In this case, you'd be better off in doing that in a database.

So, in short, if you want to do ETL, it is great. If you want to compute metrics over five years of data and tremendous volumes and aggregate this, the databases, relational databases particularly are much better at that. Now there are three concepts when it comes to this platform. The very first one is the FlowFile, you can think of this as the data. It contains the data itself, and then some attributes such as the key value pairs associated with the data. Then you have the processor, and you can think of the processor as the transformation. It applies

transformation to FlowFiles, as you can see there on the first bullet, and every transformation creates a new output from the transformation. So, you can pair these up, and you can take a processor that outputs feeds into the next processor input. As you can see there in the diagram, where you have processor 1, that's taking in data, it outputs a FlowFile 1, which is then ingested by processor 2, which outputs a FlowFile 2, and so on.

Now the last concept is that of a connector and you can think of this as a queue for the FlowFiles waiting to be processed by the processor. And one of the advantages there is that you can set priority rules. For example, you could do first first out, or you could do last in, first out, and such kind of rules. You can also have there some back pressure, that is setting some limits or setting some rules, for example limiting to 10,000 FlowFiles and stopping the queue there until you process more of them so you can go forward. So, as you can see there, between FlowFiles, processors, and connectors, you can create pretty rich pipelines to be able to process your data. And in this case, it is a very good fit to be able to do ETL, extract, transform and load.

